Groq API - Quick Start Guide

1. What is Groq?

Groq is an AI inference platform built around its custom Language Processing Unit (LPU), designed for very fast, low-latency LLM inference. Unlike GPU-based providers, Groq's hardware is optimized for sequential token generation, which translates to noticeably faster response times for chat and completion tasks.

Groq hosts several popular open-source models (Llama 3, Mixtral, Gemma, etc.) and exposes them through an OpenAI-compatible REST API, making it easy to integrate into existing projects or swap in as an alternative to other providers.

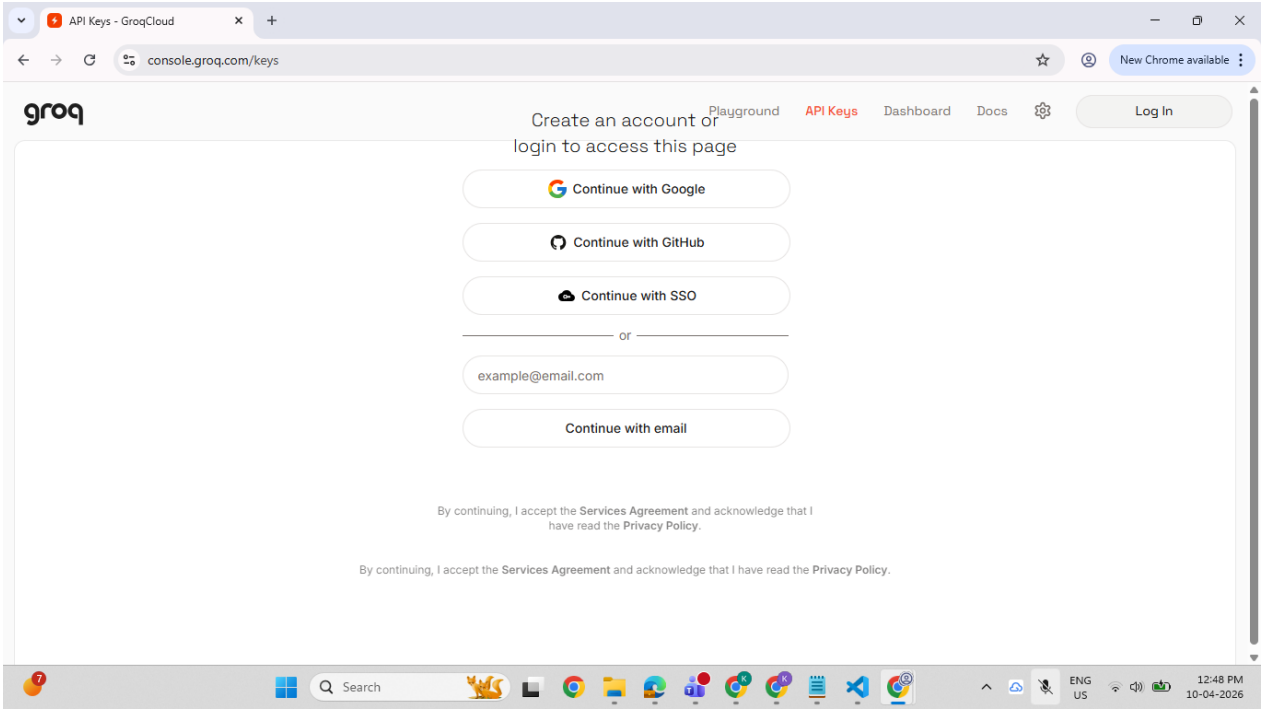
---

2. Registration & API Key Setup

Registration is free and straightforward. Follow the steps below.

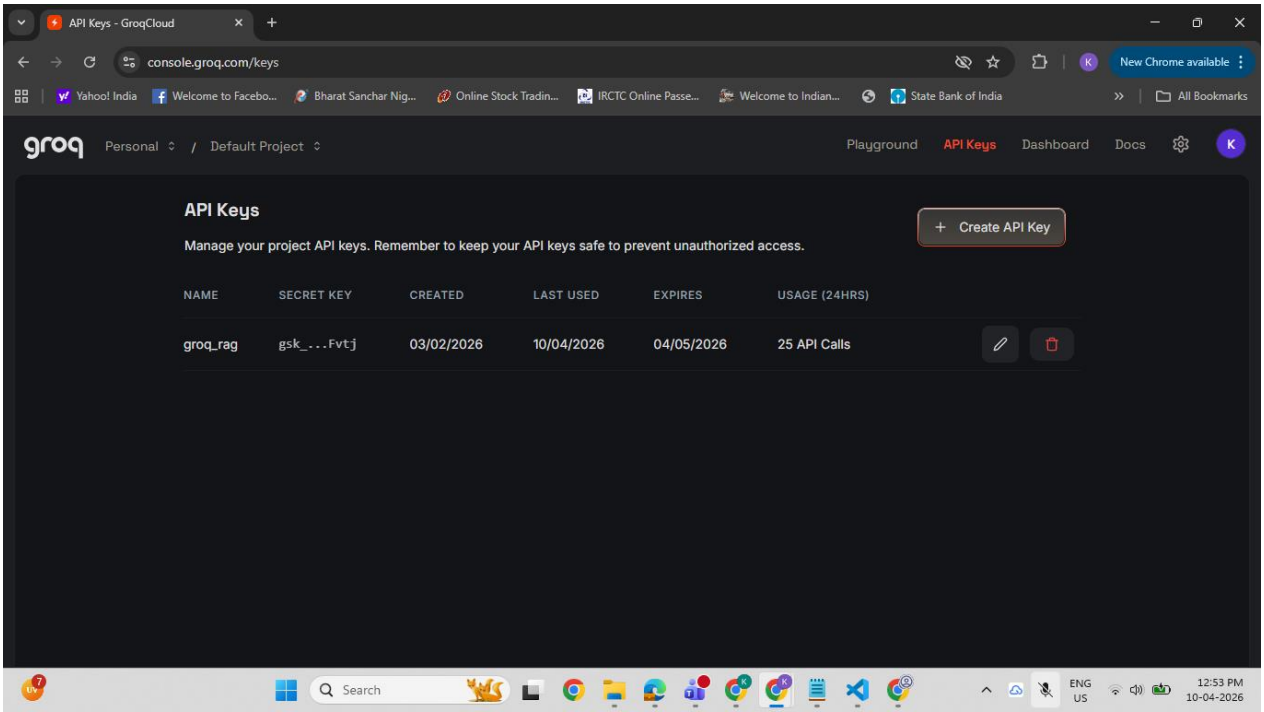
Step 1 - Create an account

Go to console.groq.com. Sign up using your email address or your GitHub account.



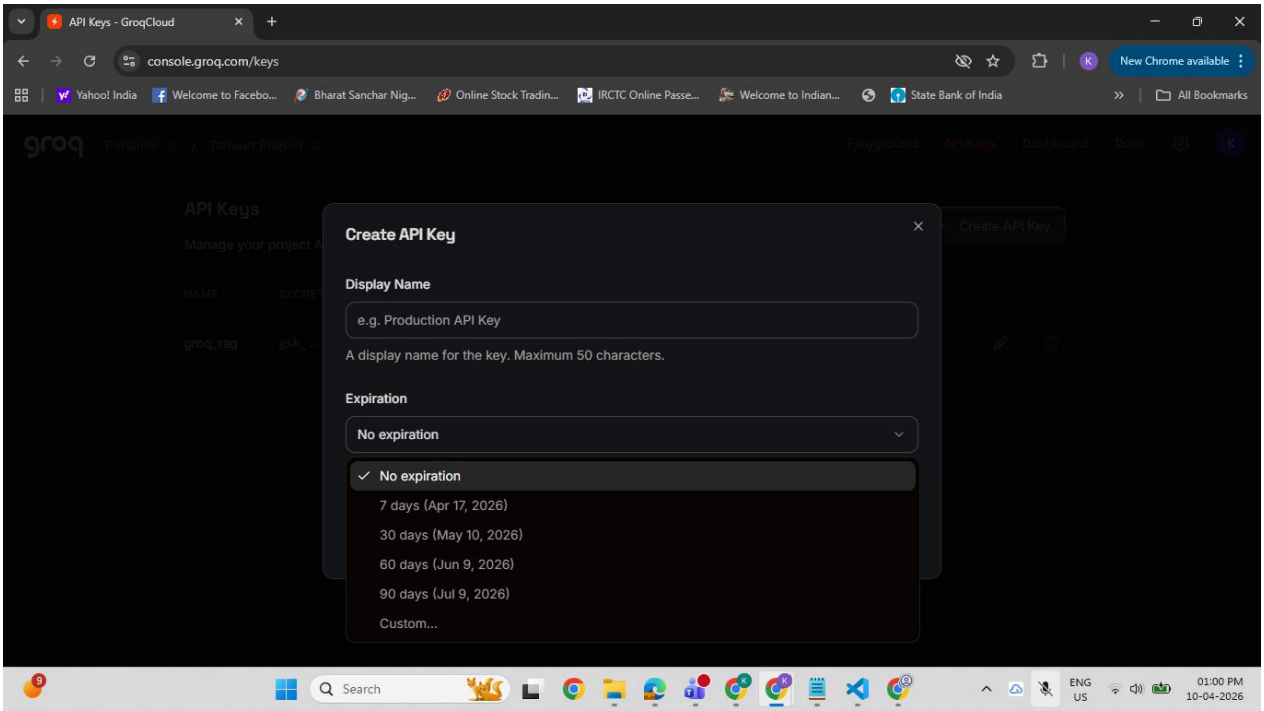
Step 2 - Navigate to API Keys

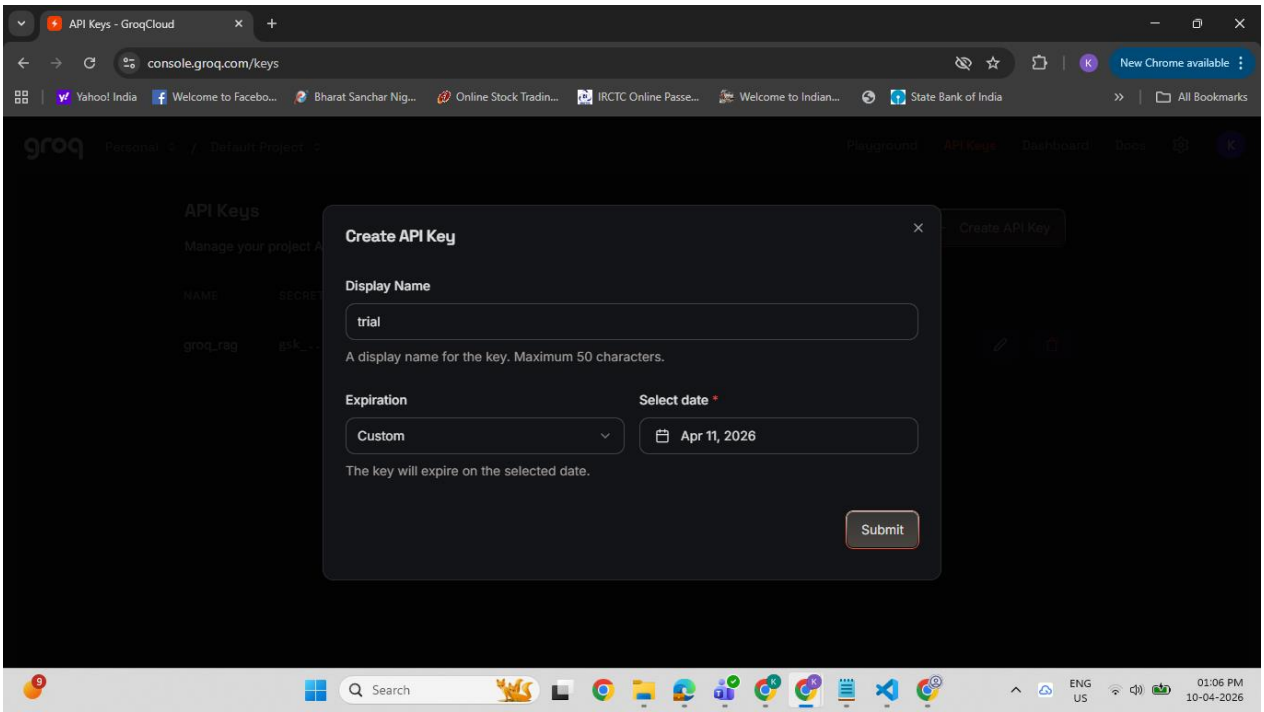
Once logged in, click on "API Keys" in the left-hand sidebar, then click "Create API Key".



Step 3 - Name & configure the key

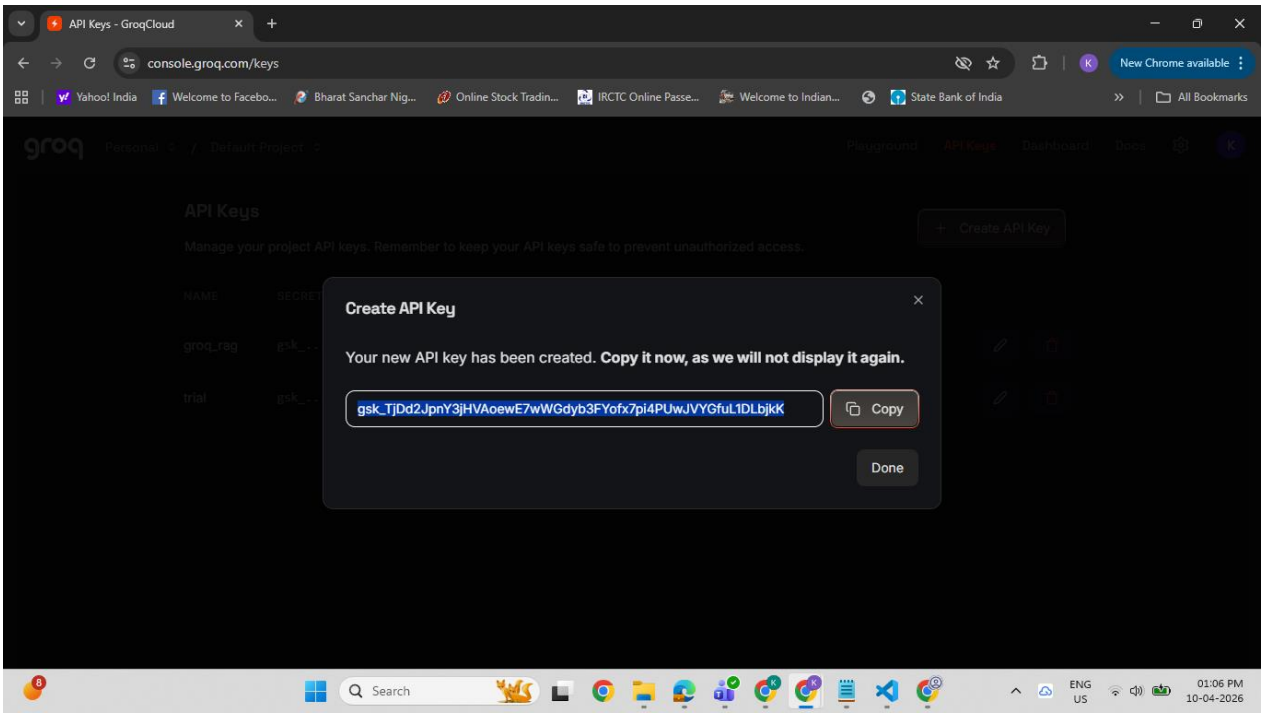
Give your key a descriptive name (e.g., poc-test) and optionally set an expiration date. For a short-term POC, leaving the expiration open is fine - just remember to revoke it afterwards.





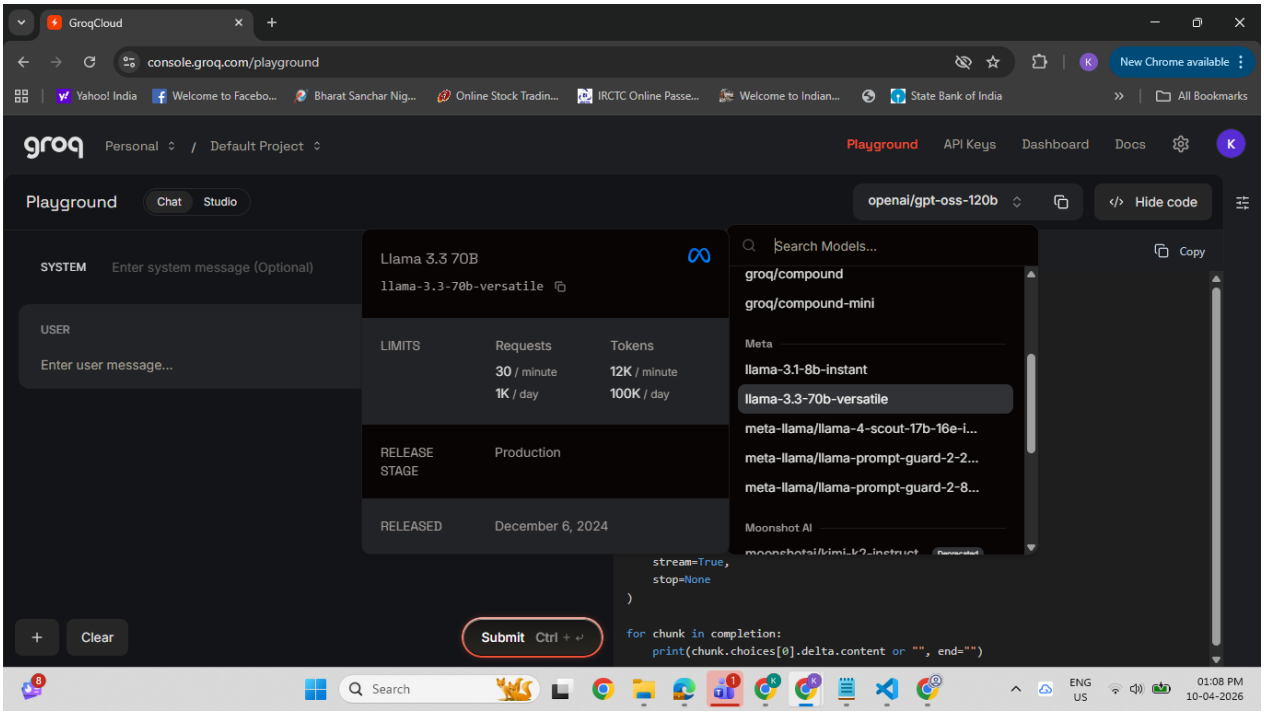
Step 4 - Copy and store the key

Copy the key immediately - it is only shown once. Store it in your project's .env file (see Section 3 for usage). Never commit this value to source control.



Step 5 - Review available models

On the Models page you can see all models available on the free tier along with their context window sizes and per-token limits. Choose the model that fits your use case.



---

3. Simple Chat Example (Python + .env)

The example below shows the minimal code needed to send a message to the Groq API using Python. The API key is loaded from a .env file so it never appears in your source code.

3.1 Install dependencies

```
pip install groq python-dotenv
```

3.2 Create your .env file

Create a file named .env in your project root (same level as your script):

```
# .env
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Add .env to your .gitignore so the key is never pushed to a repository.

```
# .gitignore
.env
```

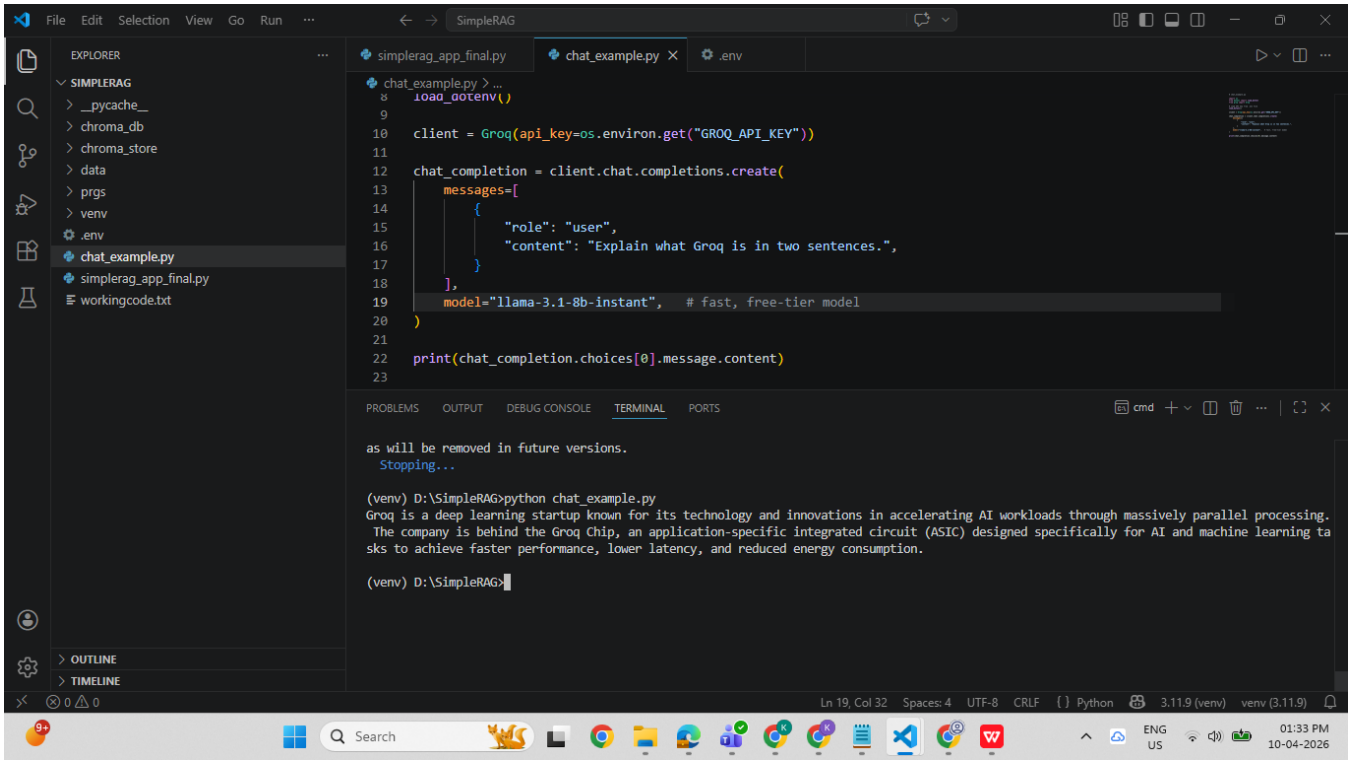
3.3 Python chat script

```python
# chat_example.py

import os
from dotenv import load_dotenv
from groq import Groq

# Load API key from .env file
load_dotenv()

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": "Explain what Groq is in two sentences.",
        }
    ],
    model=" llama-3.1-8b-instant ",    # fast, free-tier model
)

print(chat_completion.choices[0].message.content)
```

Run the script with:

```
python chat_example.py
```

The Groq SDK's interface is intentionally identical to the OpenAI Python SDK. If your team already uses OpenAI, the migration to Groq (or switching between the two) requires changing only the client import and the base URL - no other code changes are needed.



You can also set the model temperature and see the difference between the answers generated :

```
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": "Explain what Groq is in two sentences.",
        }
    ],
    model="llama-3.1-8b-instant",# fast, free-tier model
    temperature=0.3,
)
print(chat_completion.choices[0].message.content)
```

```
as will be removed in future versions.
  Stopping...

(venv) D:\SimpleRAG>python chat_example.py
Groq is a deep learning startup known for its technology and innovations in accelerating AI workloads through massively parallel processing.
 The company is behind the Groq Chip, an application-specific integrated circuit (ASIC) designed specifically for AI and machine learning ta
sks to achieve faster performance, lower latency, and reduced energy consumption.

(venv) D:\SimpleRAG>python chat_example.py
Groq is a US-based company that specializes in developing and manufacturing high-performance AI computing hardware, particularly focused on
tensor processing units (TPUs). Their products are designed to accelerate artificial intelligence and machine learning workloads, providing
significant performance improvements and energy efficiency compared to traditional computing architectures.
```

4. Free Tier Limitations

The free tier is generous for development and POC work but does have hard caps. These are enforced at the API level; once a limit is hit, requests return a 429 Too Many Requests error until the window resets.

4.1 Rate limits (as of the time of writing)

Limits are applied across three dimensions simultaneously:

- Requests per Minute (RPM) - the number of API calls you can make in any rolling 60-second window.
- Tokens per Minute (TPM) - the total input + output tokens consumed within 60 seconds.
- Tokens per Day (TPD) - a daily cap on total token consumption. This resets every 24 hours (UTC).

Each model has its own specific limits. Larger or more capable models (e.g., Llama 3 70B) have lower limits than smaller ones (e.g., Llama 3 8B). Check the Groq console "Limits" page for the most current values, as these are updated periodically.

4.2 No rate-limit issues observed during POC

During our internal POC, we did not encounter any rate-limiting errors. The free tier was more than sufficient for iterative testing and exploratory development. This is consistent with Groq's positioning - they want developers to evaluate the platform without friction.

That said, if you are planning a load test or batch-processing scenario, you should account for the TPD cap, as high-volume runs can exhaust the daily token budget faster than expected.

4.3 Other constraints

- No SLA guarantee on the free tier - availability is best-effort.
- No fine-tuning support on the free tier.
- Context window is model-dependent; most free-tier models support 8 K to 128 K tokens.
- API keys created on the free tier cannot be scoped to specific models or IPs; all restrictions are account-wide.
- Paid tiers unlock higher limits, dedicated capacity, and priority support.